

# Neural networks, from the perceptron up

Dataset: Fashion-MNIST

## How to use these notes

The primary text for this lecture is Géron, Chapter 9. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

## Contents

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>The brief, and why this is not Lecture 3</b>                 | <b>2</b> |
| 1.1      | The corpus, and what it does not contain . . . . .              | 2        |
| <b>2</b> | <b>Derivation: what a layer computes</b>                        | <b>3</b> |
| 2.1      | One layer, in four steps . . . . .                              | 4        |
| <b>3</b> | <b>From one unit to a network</b>                               | <b>5</b> |
| 3.1      | Price it before you fit it . . . . .                            | 5        |
| 3.2      | The output layer, and the loss that must pair with it . . . . . | 5        |
| <b>4</b> | <b>Fitting it, and reading what happened</b>                    | <b>6</b> |
| <b>5</b> | <b>Ten fits, and three things they do not tell you</b>          | <b>6</b> |
| <b>6</b> | <b>One number over ten classes</b>                              | <b>7</b> |
| <b>7</b> | <b>Four walls, and they are all the same wall</b>               | <b>8</b> |

|   |              |   |
|---|--------------|---|
| 8 | The notebook | 8 |
| 9 | Exercises    | 9 |
|   | Summary      | 9 |

---

## 1 The brief, and why this is not Lecture 3

A document-processing bureau scans incoming items and routes each to one of ten queues. Items arrive as small greyscale scans, already cropped and centred; a human currently sorts them, and the operator wants to know how often a machine would be right. Misrouting is recoverable but expensive — the item goes to the wrong desk and comes back a day later.

Ten classes, one label each, no ranking, no probability requirement stated. Write that down: it decides the output layer.

| Lectures 3–4                                | Today                                              |
|---------------------------------------------|----------------------------------------------------|
| Two classes                                 | ten classes                                        |
| Wildly imbalanced                           | exactly balanced                                   |
| Accuracy was a trap                         | accuracy is defensible — we will check, not assume |
| The question was where to put the threshold | the question is what function to fit               |

### The habit worth keeping

Not “accuracy is bad”. *Count the classes before you choose.* Here every class holds exactly 6,000 images, so “always  $k$ ” scores 10.0% and accuracy cannot hide a useless model.

### 1.1 The corpus, and what it does not contain

Fashion-MNIST: 60,000 training and 10,000 test images,  $28 \times 28$  greyscale, ten balanced classes. We carve 5,000 validation images out of the training half with a fixed seed — 55,000 / 5,000 / 10,000 — because we are about to try ten configurations, and Lecture 2 established what happens to a number chosen on the set it is reported on.

What the data does not contain: *no colour*, so a red shirt and a blue shirt are the same image; *no scale*, since every garment fills the frame, so size cannot distinguish a sandal from a boot; and *no context*, one centred item on a black background. Every one of those is a simplification the bureau’s real scans will not honour, so a number measured here is an upper bound on what the same method gives on the archive — and the report should say so.

### A pixel is a feature, and that costs something

We flatten each image into 784 numbers and hand it over as an ordinary table. The model is told nothing about which pixels are neighbours: shuffle all 784 columns with one fixed permutation and it learns exactly as well, though a human could not read a single shuffled image.

That is a real loss. Repairing it is four lectures away. Today we pay it, and it is the honest starting point.

Dividing the pixels by 255 is not Lecture 2's leak. `StandardScaler` estimates a mean and a variance from data and must be fitted after the split;  $x/255$  estimates nothing — it divides by a constant that is a property of the file format.

## 2 Derivation: what a layer computes

The vocabulary is historical. McCulloch and Pitts modelled a biological neuron that fires when enough of its inputs fire, which is where *neuron*, *activation* and *firing* come from. It is not an argument for anything; modern networks resemble brains about as closely as aeroplanes resemble birds.

A **threshold logic unit** computes

$$z = \mathbf{w}^T \mathbf{x} + b, \quad \hat{y} = \text{step}(z).$$

A weighted sum and a threshold — and the weighted sum is Lecture 2's linear model wearing a different name. Rosenblatt trained it with

$$w_{i,j}^{\text{next}} = w_{i,j} + \eta (y_j - \hat{y}_j) x_i,$$

which converges only if the classes are linearly separable. *No loss function appears anywhere in that rule*, which is the tell: this is not gradient descent, and it does not generalise.

And a single unit draws one straight line. XOR needs two, so no TLU computes it — Minsky and Papert said so in 1969 and the field stopped for fifteen years. The repair is embarrassingly simple: stack them.

## 2.1 One layer, in four steps

### From one unit to a mini-batch

A single unit computes  $z = \mathbf{w}^T \mathbf{x} + b$ : two numbers per unit and no more, a direction and how far along it to sit.

Stack  $n$  units side by side and the weights become a matrix,  $\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$  with one column per unit. The layer is *one matrix product*, which is why a tuned BLAS matters at all.

Feed a whole mini-batch by making  $\mathbf{X}$  one row per instance:  $\mathbf{Z} = \mathbf{XW} + \mathbf{b}$ , with shapes  $(m \times d)(d \times n) \rightarrow (m \times n)$  and  $\mathbf{b}$  broadcasting along the rows.

Apply a non-linearity elementwise:

$$\mathbf{H} = \phi(\mathbf{XW} + \mathbf{b}).$$

Elementwise, so it changes no shape — the arithmetic above is untouched.

### Now remove $\phi$ , and watch the stack collapse

Compose two layers with nothing between them:

$$(\mathbf{XW}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 = \mathbf{X}(\mathbf{W}_1\mathbf{W}_2) + (\mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2).$$

Matrix product is associative, so the pair collapses to  $\mathbf{XW}' + \mathbf{b}'$  — one affine map. A stack of linear layers, however deep, is a linear model.

### So $\phi$ is not a refinement

Without it there is no second layer at all. Two hundred stacked layers would still be a single affine map, and that is one line of algebra — it is the reason activation functions exist.

The step function had to go for a different reason: gradient descent needs a slope, and the step's derivative is zero everywhere it exists.

| Activation | Range         | Cost           | Problem                |
|------------|---------------|----------------|------------------------|
| Sigmoid    | $(0, 1)$      | an exponential | saturates at both ends |
| tanh       | $(-1, 1)$     | an exponential | saturates at both ends |
| ReLU       | $[0, \infty)$ | a comparison   | flat below zero        |

$\max(0, z)$  is fast, does not saturate above, and in practice trains better. “Flat below zero” is a real defect with a name — dying ReLUs — diagnosed in Lecture 11, not here.

### What the derivation buys you

A layer is one matrix product and one elementwise function; everything else is bookkeeping. The parameter count is  $d \times n + n$  per layer, so you can *price* a network before fitting it. Depth is only worth having because  $\phi$  is there. And the shapes compose, which is what makes a mini-batch free.

Lecture 10 differentiates exactly this expression, once per layer, backwards. Lecture 11 asks what happens to the variance of  $\mathbf{H}$  as the stack gets deeper, and answers with the same two objects.

### 3 From one unit to a network

#### 3.1 Price it before you fit it

```
784 x 300 + 300 = 235,500   layer 1
300 x 100 + 100 = 30,100    layer 2
100 x 10 + 10 = 1,010       output
-----
266,610   total
```

266,610 parameters fitted from 55,000 images — nearly five parameters per training image. Lecture 2's failure condition for least squares was  $m < n$ , more parameters than instances. We are past it comfortably, and the model will still work. Understanding why is Lectures 13 and 14.

#### 3.2 The output layer, and the loss that must pair with it

Ten mutually exclusive classes, so ten output units and a softmax across them:

$$\hat{p}_k = \frac{\exp(z_k)}{\sum_{j=1}^{10} \exp(z_j)}.$$

Every output is positive and they sum to 1, and the largest logit gives the largest probability, so  $\operatorname{argmax}$  is unchanged by the softmax. The loss is cross-entropy,

$$L = - \sum_{k=1}^{10} y_k \log \hat{p}_k = - \log \hat{p}_{\text{true}},$$

since the target is one-hot and nine of the ten terms are zero. Chapter 4 called this log loss when there were two classes; it is the same object.

| Task              | Output units  | Output activation | Loss                 |
|-------------------|---------------|-------------------|----------------------|
| Binary            | 1             | sigmoid           | binary cross-entropy |
| Multilabel binary | one per label | sigmoid           | binary cross-entropy |
| Multiclass        | one per class | softmax           | cross-entropy        |

Ours is the third row. The distinction between rows two and three is not cosmetic: multilabel asks ten independent yes/no questions, multiclass asks one ten-way question, and softmax couples the outputs where ten sigmoids do not.

Those tables answer three questions whose answers are *forced* rather than chosen — how many output units (the shape of the answer), which activation (the range the answer must live in), and which loss (what a wrong answer costs).

#### The pairing is not optional

Softmax with mean squared error trains, slowly and badly: the gradient through a saturated softmax nearly vanishes, and MSE does nothing to undo it. Cross-entropy is the loss that cancels the softmax derivative, which Lecture 17 derives. Until then, take the pairing from the table.

Two names arrive here and are cashed later. **Backpropagation** is one forward pass to compute the loss, one backward pass to compute every partial derivative, and one descent step — and the backward pass costs about as much as the forward one *no matter how many parameters there are*. 266,610 parameters, one number of interest, and a single sweep gets all the derivatives; that claim is derived at the top of the next lecture and is the whole reason any of this is feasible. And **Adam** keeps running averages of the gradient and its square and scales each parameter's step by the second, which makes it far less sensitive to the initial learning rate than plain SGD — though less sensitive is not insensitive, as the sweep below shows.

## 4 Fitting it, and reading what happened

| Quantity            | Measured |
|---------------------|----------|
| Training images     | 55,000   |
| Epochs              | 20       |
| Wall clock, CPU     | 125 s    |
| Validation accuracy | 88.94%   |
| Test accuracy       | 88.02%   |
| Baseline            | 10.0%    |

Hold on to the wall clock — it is the number the next lecture repairs.

Three words get confused, and Scikit-Learn does not help. A **batch** is the instances used for one weight update (128 here); a **step** is one weight update (430 per epoch); an **epoch** is one complete pass over the training set (20 of them). Scikit-Learn calls the number of epochs `max_iter`. It is not iterations. Read the docstring, not the parameter name.

The learning curves separate and stay separated, and that gap is overfitting — Lecture 2's table drawn as lines. The loss falls smoothly, so the optimiser is working; training accuracy keeps climbing, so the model has capacity to spare; validation accuracy flattens, so the extra capacity is being spent on memorising. And the curve answers a question the number cannot: would more epochs help? Here, no — the validation curve stopped moving before the budget ran out, and without the curve you would have guessed.

| Repair for that gap                  | Available today                  | Taught in               |
|--------------------------------------|----------------------------------|-------------------------|
| Stop when validation stops improving | <code>early_stopping=True</code> | Lecture 5, applied here |
| Penalise large weights               | <code>alpha</code>               | Lecture 5               |
| Dropout                              | not available                    | Lecture 10, then 11     |
| Batch normalisation                  | not available                    | Lecture 11              |
| More data, or augmented data         | not available                    | Lecture 13              |

Three of the five are not reachable from this library at all — and they all need a hook inside the training loop.

## 5 Ten fits, and three things they do not tell you

Five architectures and five learning rates, on a 10,000-image subset so they fit in the lecture.

| Hidden layers | Parameters | Validation accuracy | Seconds |
|---------------|------------|---------------------|---------|
| 30            | 23,860     | 84.66%              | 169     |
| 100           | 79,510     | 86.16%              | 225     |
| 300           | 238,510    | 86.20%              | 347     |
| 300, 100      | 266,610    | 84.80%              | 555     |
| 300, 200, 100 | 316,810    | 84.24%              | 841     |

| Learning rate       | Validation accuracy |
|---------------------|---------------------|
| 0.0001              | 84.28%              |
| 0.0003              | 86.22%              |
| 0.001 — the default | 84.80%              |
| 0.003               | 85.76%              |
| 0.01                | 84.20%              |

On this subset *depth loses*: one hidden layer of 300 beats both deeper networks, and the deepest is the worst and the slowest. And the library’s default learning rate is not the best value on the grid.

#### Neither knob dominates — on this grid, at this scale

The five architectures span about two accuracy points; so do the five learning rates. The notebook sweeps 6,000 images rather than 10,000 and gets a *different* answer: there the learning rate spans 2.9 points and the architectures 1.3, and even the winning rate changes. Two runs of the same sweep, one lecture, two conclusions. Which is why “the learning rate matters more” is a claim about a grid and a scale, never about the world — and why a sweep must always be reported with the size it was run at.

Ten fits is also not a search: ten points from a space with five axes, varied one axis at a time with everything else held fixed. You have not optimised anything; you have looked at ten places.

#### And the ranking does not transfer

| Run                                        | Images | Epochs | Accuracy          |
|--------------------------------------------|--------|--------|-------------------|
| Best in the sweep — 300, at lr 0.001       | 10,000 | 12     | 86.20% validation |
| The headline model — 300, 100, at lr 0.001 | 55,000 | 20     | 88.02% test       |

The architecture the sweep ranked *third of five* is the one that wins with all the data. A deeper network needs more data to pay for its extra parameters, so ranking architectures on a fifth of your data systematically prefers the shallow ones — and never finds out.

## 6 One number over ten classes

Split it. Lecture 3 built the confusion matrix for two classes and the object generalises without change: row  $i$ , column  $j$  counts the images of class  $i$  the model called class  $j$ .

Recall on *Shirt* is 74.1%, against 98.1% on *Sneaker*. Shirts are called Pullover 10.9% of the time, T-shirt/top 7.8%, and Coat 4.6% — exactly the three classes anyone would name from the image grid, and Coat is nearly as bad at 74.8%. *The model is not confused about anything a person would find easy.*

#### A defensible report

“Overall it is right 88.0% of the time. But it is right 74.1% of the time on Shirt, and almost all of those errors go to three visually similar queues. If those three desks are adjacent, the practical error rate is much lower than the headline. If they are in different buildings, it is much worse.”

The headline accuracy does not answer the operator’s question. The per-class breakdown does, and it took one extra function call.

## 7 Four walls, and they are all the same wall

| What you want   | What happens                                                                                                                       |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Make it faster  | there is no device=. Scikit-Learn does not support GPUs and says so in its own FAQ — a deliberate scope decision, not an oversight |
| Change the loss | loss="mae" raises TypeError. Cross-entropy is hard-coded                                                                           |
| See a gradient  | [a for a in dir(clf) if "grad" in a.lower()] is empty. 266,610 partial derivatives were computed on every batch and none was kept  |
| Stop part-way   | partial_fit is the finest handle there is, and one call is one complete pass over everything you hand it                           |

Each of these matters for an ordinary reason, not an exotic one. A weighted loss is one line in a framework that lets you write one, and the operator saying “calling a coat a shirt is cheap, calling a bag a sandal is expensive” is the normal case. Lecture 11 builds a twenty-layer network whose loss barely moves, and the diagnosis is a per-layer gradient statistic you cannot compute here.

#### The problem underneath all four

`fit()` contains a `for` loop over epochs and batches that you did not write, cannot see, and cannot enter. Every one of the four walls is that one wall.

This is not a criticism of Scikit-Learn — `MLPClassifier` is documented as a convenience, and for a tabular problem with a standard loss it is the right tool. It is the wrong tool from here on, and now you know precisely why.

## 8 The notebook

#### What to change

Remove the activation — set it to identity — and refit. Accuracy collapses towards what a linear model gets, which is the collapse derived in §2, measured.



## 9 Exercises

1. Show that a stack of dense layers with identity activations is a single affine map, and state what that implies about depth. [5]
2. A network has 784 inputs, hidden layers of 300 and 100 units, and 10 outputs. Compute the parameter count, and say which layer holds most of it and why. [4]
3. A task has ten mutually exclusive classes. Give the number of output units, the output activation and the loss, and explain why the same answer would be wrong for ten independent yes/no labels. [4]
4. A five-point architecture sweep on a fifth of the data ranks a deeper network third of five; trained on all the data it wins. Explain the mechanism, and say what it implies about tuning on subsamples. [5]
5. A model reports 88.0% accuracy on ten balanced classes. State one further measurement you would make before reporting it to the stakeholder, and what decision it would inform. [4]

## Summary

A layer is one matrix product and one elementwise non-linearity,  $\mathbf{H} = \phi(\mathbf{XW} + \mathbf{b})$ , and without  $\phi$  any stack collapses to a single affine map by associativity — so depth exists only because the non-linearity does. The parameter count  $d \times n + n$  per layer lets you price a network before fitting it: 266,610 for ours, from 55,000 images.

The architecture tables answer three forced questions — how many output units, which activation, which loss — and the loss must pair with the activation rather than being chosen beside it. The network reached 88.02% test accuracy against a 10.0% baseline in 125 seconds on a CPU.

Ten hand fits gave three lessons, none of them about the winner. Depth lost on 10,000 images; neither knob dominated *on that grid at that scale*, and the notebook's 6,000-image sweep reaches a different conclusion from the same code; and the architecture ranked third of five is the one that wins with all the data, because a deeper network needs more data to pay for its parameters.

Then four things `MLPClassifier` will not do — run on a GPU, change the loss, expose a gradient, or stop inside an epoch — which are all the same thing: the training loop was written for you, and you cannot enter it.